

Лабораторная Работа №7

Введение в работу с данными

Козлов В.П.

Российский университет дружбы народов им. Патриса Лумумбы, Москва, Россия

- Козлов Всеволод Павлович
- НФИбд-02-22
- Российский университет дружбы народов
- [1132226428@pfur.ru]

Выполнение лабораторной работы

Основной целью работы является специализированных пакетов Julia для обработки данных.

1. Используя Jupyter Lab, повторите примеры из раздела 7.2.
2. Выполните задания для самостоятельной работы (раздел 7.4).

Считывание данных

```
[3]: # 7.2.1.1 Считывание данных из CSV
P = CSV.File("programminglanguages.csv") |> DataFrame

# Функция для поиска года создания языка (без учёта регистра)
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:, 2]) .== lowercase(language))
    return P[loc, 1]
end

# Пример вызова функции
language_created_year_v2(P, "julia")

[3]: 2012

[4]: # 7.2.1.2 Запись данных в файл
CSV.write("programming_languages_data2.csv", P)

[4]: "programming_languages_data2.csv"
```

Figure 1: Считывание данных

Описание датасета

```
[5]: # 7.2.1.3 Словарь
dict = Dict{Int64, Vector{String}}{()}
for i in 1:size(P, 1)
    year = P[i, 1]
    lang = P[i, 2]
    if year in keys(dict)
        push!(dict[year], lang)
    else
        dict[year] = [lang]
    end
end
dict[2003]
```

```
[5]: 2-element Vector{String}:
     "Groovy"
     "Scala"
```

```
[6]: # 7.2.1.5 RDatasets
iris = dataset("datasets", "iris")
typeof(iris)
describe(iris)
```

```
[6]: 5x7 DataFrame
```

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64

Figure 2: Описание датасета

Missing values

```
[8]: # 7.2.1.6 Работа с Missing Values
foods = ["apple", "cucumber", "tomato", "banana"]
calories = [missing, 47, 22, 105]
prices = [0.85, 1.6, 0.8, 0.6]
dataframe_calories = DataFrame(item = foods, calories = calories)
dataframe_prices = DataFrame(item = foods, price = prices)

# Исправленная строка: используем innerjoin вместо join
DF = innerjoin(dataframe_calories, dataframe_prices, on = :item)

mean(skipmissing(calories))
```

```
[8]: 58.0
```

Figure 3: Missing values

Кластеризация через k-mean

```
[11]: # 7.2.2.1 Кластеризация методом k-средних
houses = CSV.File("houses.csv") |> DataFrame
filter_houses = houses[houses[l, :sq_ft] .> 0, :]
X = filter_houses[l, [:latitude, :longitude]]

# Исправленная строка: сначала получаем значения, затем конвертируем
X = Matrix{Float64}(X) # Вместо convert(Matrix{Float64}, X)

X = X'
k = length(unique(filter_houses[l, :zip]))
C = kmeans(X, k)
df_clusters = DataFrame(cluster = C.assignments, city = filter_houses[l, :city],
                        latitude = filter_houses[l, :latitude], longitude = filter_houses[l, :longitude],
                        zip = filter_houses[l, :zip])

clusters_figure = plot(legend = false)
for i in 1:k
    clustered_houses = df_clusters[df_clusters[l, :cluster] .== i, :]
    scatter!(clusters_figure, clustered_houses[l, :latitude], clustered_houses[l, :longitude], markersize = 4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

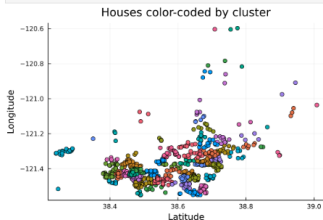


Figure 4: Кластеризация через k-mean

```
[12]: # 7.2.2.2 Метод k ближайших соседей
      knearest = 10
      id = 70
      point = X[:, id]
      kdtree = KDTree(X)
      idxs, dists = knn(kdtree, point, knearest, true)
      scatter(filter_houses[!, :latitude], filter_houses[!, :longitude])
      scatter!(filter_houses[idxs, :latitude], filter_houses[idxs, :longitude])
      cities = filter_houses[idxs, :city]

[12]: 10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
      "SACRAMENTO"
      "ELK GROVE"
      "SACRAMENTO"
      "SACRAMENTO"
      "SACRAMENTO"
      "SACRAMENTO"
      "ELK GROVE"
      "ELK GROVE"
      "ELK GROVE"
      "ELK GROVE"
```

Figure 5: KNN

```

0]: # 7.2.2.3 Метод главных компонент (PCA)
using MultivariateStats, Plots

# Подготовка данных
F = filter_houses[!, [:sq_ft, :price]]
println("Размер DataFrame F: ", size(F))

# Конвертируем в матрицу
F_matrix = Matrix{Float64}(F)' # (2, n_samples)
println("Размер F_matrix (транспонированный): ", size(F_matrix))

# Обучаем PCA
M = MultivariateStats.fit(PCA, F_matrix; maxoutdim=2)
println("PCA обучена успешно")

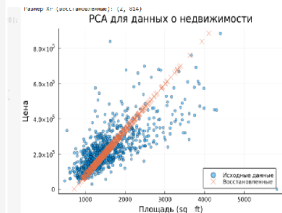
# Используем transform из MultivariateStats
P = MultivariateStats.transform(M, F_matrix) # Проекция
println("Размер P (проекция): ", size(P))

# Восстановление
Xr = MultivariateStats.reconstruct(M, P)
println("Размер Xr (восстановленные): ", size(Xr))

# Визуализация
scatter(F_matrix[1, :], F_matrix[2, :],
    label="Исходные данные",
    alpha=0.5,
    markersize=3,
    title="PCA для данных о недвижимости",
    xlabel="Площадь (sq_ft)",
    ylabel="Цена",
    legend=:bottomright)

scatter!(Xr[1, :], Xr[2, :],
    label="Восстановленные",
    alpha=0.7,
    markersize=5

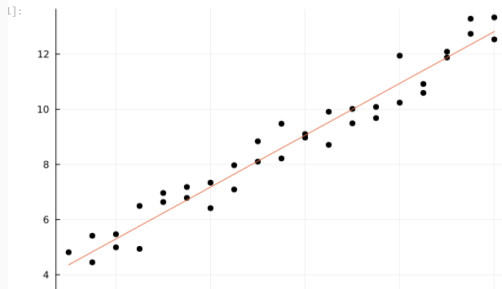
```



Линейная регрессия

```
function find_best_fit(xvals, yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals, yvals)
    a = r * stdy / stdx
    b = meany - a * meanx
    return a, b
end

xvals = repeat(1:0.5:10, inner = 2)
yvals = 3 + xvals + 2 .* rand(length(xvals)) - 1
a, b = find_best_fit(xvals, yvals)
ynew = a .* xvals + b
scatter(xvals, yvals, color = :black, leg = false)
plot!(xvals, ynew)
```



```
[22]: # Задание 2: Кластеризация набора Iris
iris = dataset("datasets", "iris")
iris_matrix = Matrix(iris[, 1:4])
k_iris = 3
C_iris = kmeans(iris_matrix, k_iris)
scatter(iris_matrix[1, :], iris_matrix[2, :], group = C_iris.assignments, legend = false)
```

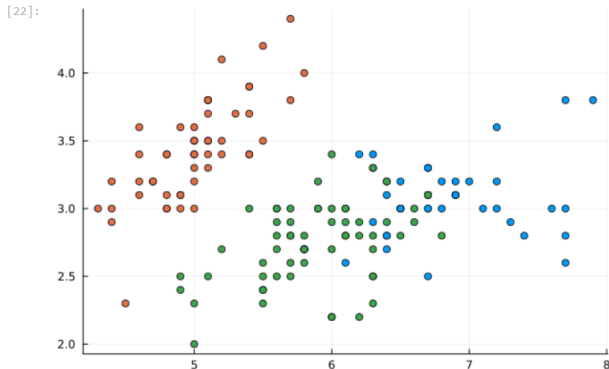


Figure 8: Датапоинты

График регрессии

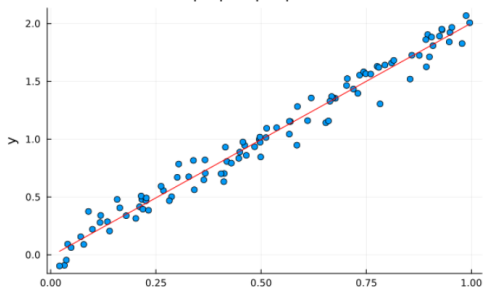
```
[23]: # Задание 3: Линейная регрессия (часть 1)
X = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000)
X2 = hcat(ones(size(X, 1)), X)
a_hat = X2 \ y
println("Оценка коэффициентов: ", a_hat)
```

Оценка коэффициентов: [0.003255403542677329, 0.19813363923539568, 0.5488655528083908, 0.8750372045153417]

```
[24]: # Задание 3: Линейная регрессия (часть 2)
X = rand(100)
y = 2 .* X + 0.1 .* randn(100)
a, b = find_best_fit(X, y)
scatter(X, y, leg = false, title = "График регрессии", xlabel = "X", ylabel = "y")
plot!(X, a .* X .+ b, color = :red)
```

[24]:

График регрессии



Графики акций

```
# Задание 4: Модель ценообразования биномиальных опционов
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    h = T / n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p_star = (exp(r * h) - d) / (u - d)
    path = zeros(n + 1)
    path[1] = S
    for t in 2:(n + 1)
        path[t] = path[t - 1] * (rand() < p_star ? u : d)
    end
    return path
end

S, T, n, sigma, r = 100.0, 1.0, 10000, 0.3, 0.08
path = createPath(S, r, sigma, T, n)
plot(0:n, path, title = "Траектория цены акции", xlabel = "Время", ylabel = "Цена", log = false)

# Генерация 10 траекторий
paths = [createPath(S, r, sigma, T, n) for _ in 1:10]
plot(0:n, paths, title = "10 траекторий цен акций", xlabel = "Время", ylabel = "Цена", legend = false)
```

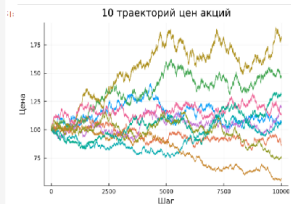


Figure 10: Графики акций

Освоил специализированные пакеты Julia для обработки данных.